

基础软件开发技术实践

智能软件工程实验报告

实验一：代码审查意见挖掘与需求理解

姓名	廖正强	院系	计算学部
学号	2023112920	任课教师	刘佳琨
指导教师	刘佳琨	实验地点	格物 213
实验时间	2026/7/3		
实验得分		研讨得分	
报告得分		实验总分	

一、实验目的

本实验是“代码审查（Code Review）”研究系列的第一个实验，旨在构建一个包含丰富信息的代码审查数据集，该数据集将作为后续所有实验的唯一数据来源。

具体目标包括：

- (1) 熟悉 GitHub Pull Request 的组织形式，理解 PR、Review、Commit 之间的关系；
- (2) 通过 GitHub API 获取 5 个大型开源项目的 Pull Request 数据；
- (3) 获取每条 PR 对应的 Code Review 信息（Review 决策、Reviewer、Inline Comments）；
- (4) 获取代码修改内容（修改文件、修改函数、Diff、Commit 信息）；
- (5) 提取结构化特征（PR 长度、文件数、Reviewer 数、Comment 数等）；
- (6) 对数据集进行统计分析和可视化，为后续的 Merge Prediction 和 Review Comment Generation 任务奠定数据基础。

二、实验内容

2.1 数据采集

目标项目选择：选取 5 个具有代表性的大型开源项目：Kubernetes、PyTorch、VS-Code、TensorFlow、React。这些项目覆盖了容器编排、深度学习框架、代码编辑器、科学计算和前端框架等不同领域，PR 数量充足，审查流程规范。

数据获取流程：使用 Python 编写数据采集脚本，通过 GitHub REST API v3 进行数据拉取。每条 PR 采集 5 个端点的数据：

- 步骤 1: 获取 **PR 基本信息** (GET /repos/{owner}/{repo}/pulls/{number}): PR ID、标题、正文、作者、创建时间、合并状态、标签列表。
- 步骤 2: 获取 **Code Review** (GET .../pulls/{number}/reviews): Reviewer、审查决策 (APPROVED / CHANGES_REQUESTED / COMMENTED)、审查时间、审查评论正文。
- 步骤 3: 获取 **Inline Review Comments** (GET .../pulls/{number}/comments): 代码行级评论，包含文件路径、行号和评论内容。
- 步骤 4: 获取**代码修改** (GET .../pulls/{number}/files): 修改文件列表、增删行数、Unified Diff Patch。
- 步骤 5: 获取 **Commit 信息** (GET .../pulls/{number}/commits): Commit Message 列表。

采集策略：按更新时间降序排列，每个仓库获取 300 条 PR (总计 1500 条)。采用 3 线程并行采集，每条 PR 内部 5 个 API 调用也并行执行，采集耗时约 15 分钟。配备断点续传机制，支持中断后自动恢复。API 调用共计约 7500 次，使用 GitHub Personal Access Token 认证 (5000 次/小时限制)，配合限速控制器确保稳定运行。

2.2 特征提取

对每条 PR 提取以下特征：

AI 检测方法：通过关键词匹配判断 PR 是否涉及 AI。检查 Review 正文、Inline Comment 正文和 PR 标签中是否包含 AI 相关关键词 (如 copilot、chatgpt、claude、gpt、llm、codex 等)。AI 生成代码的检测同理，匹配 PR 正文和标签中的 AI 生成关键词 (如 ai-generated、generated by ai、gpt- 等)。

修改函数提取：解析 Unified Diff 格式的 Patch，从 Hunk Header (@@ -a,b +c,d @@ context) 中提取上下文函数名，同时匹配新增行中的函数定义模式 (支持 Python、JavaScript、Go、C/C++、Rust 等语言)。

表 1: 特征字段说明

类别	特征	来源
PR 基本信息	pr_id, pr_number, repo, title, body	PR 详情 API
PR 统计量	pr_length (标题 + 正文字符数)	计算得出
合并信息	is_merged, merge_status	PR 详情 API
代码修改	num_changed_files, total_additions, total_deletions, modified_functions, num_commits, commit_messages	Files API Diff 解析 Commits API
审查信息	num_reviewers, num_review_comments, num_inline_comments, review_decision	Reviews API Comments API
元标签	num_labels, label_names, has_ai_reviewer, has_ai_generated_code	Labels 字段 关键词匹配

三、实验结果

要求：将实验获得的结果进行描述，基本内容包括：

3.1 数据集概况

成功构建包含 **1500 条 PR** 的结构化数据集 (CSV + JSON)，涵盖 5 个大型开源项目，每个项目 300 条。数据集包含 26 个字段，覆盖 PR 基本信息、代码审查、代码修改和元标签四大类别。

3.2 总体统计分析

表 2: 数据集总体统计

指标	数值	占比
总 PR 数	1500	100%
已合并 (Merged)	948	63.20%
未合并	552	36.80%
含 AI Reviewer	459	30.60%
含 AI 生成代码	103	6.87%

3.3 各仓库对比分析

表 3: 各仓库统计指标对比

仓库	PR 数	合并率	均评论	均审阅人	均长度
facebook/react	300	48.7%	1.04	0.71	2586.5
kubernetes/kubernetes	300	76.3%	2.93	1.24	2239.7
microsoft/vscode	300	91.7%	2.78	2.48	1461.4
pytorch/pytorch	300	4.7%	1.63	1.25	1620.6
tensorflow/tensorflow	300	94.7%	0.04	0.04	636.5

3.4 数值特征分布

各数值特征的描述性统计如下：

- **PR 长度：** mean=1709, median=736, std=3685, range=[15, 66105]。分布呈右偏态。
- **修改文件数：** mean=20.1, median=2.0, range=[0, 3000]。大多数 PR 仅修改少量文件。
- **Reviewer 数：** mean=1.14, median=1.0, range=[0, 7]。
- **Review Comment 数：** mean=1.68, median=1.0, range=[0, 35]。
- **标签数：** mean=3.61, median=1.0, range=[0, 41]。

3.5 合并 vs 未合并对比

合并的 PR 与未合并的 PR 在特征上呈现明显差异：

- 合并 PR 平均长度（1360）明显短于未合并 PR（2309），说明简洁的 PR 更易被接受。
- 合并 PR 平均修改文件数（7.5）远少于未合并 PR（41.7），小步提交更容易通过审查。
- 合并 PR 平均 Reviewer 数（1.34）高于未合并 PR（0.80），更多的审查者参与可能意味着 PR 质量更高。

3.6 可视化分析

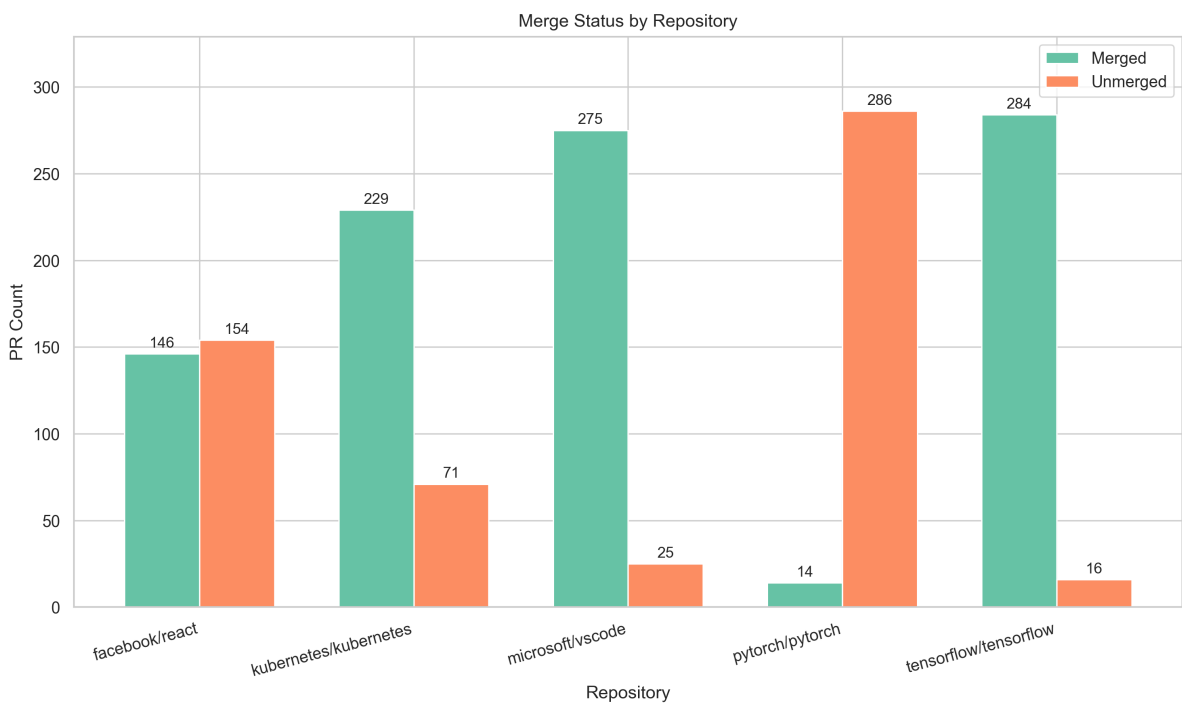


图 1: 各仓库合并状态对比柱状图

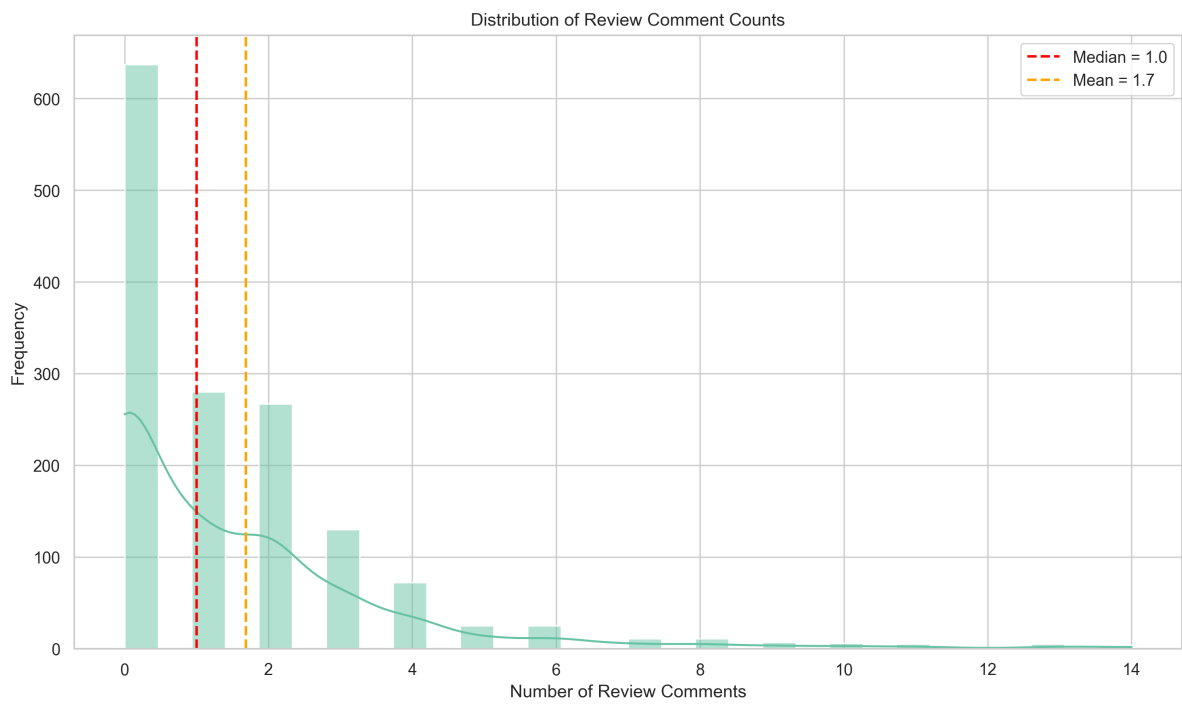


图 2: Review Comment 数量分布直方图

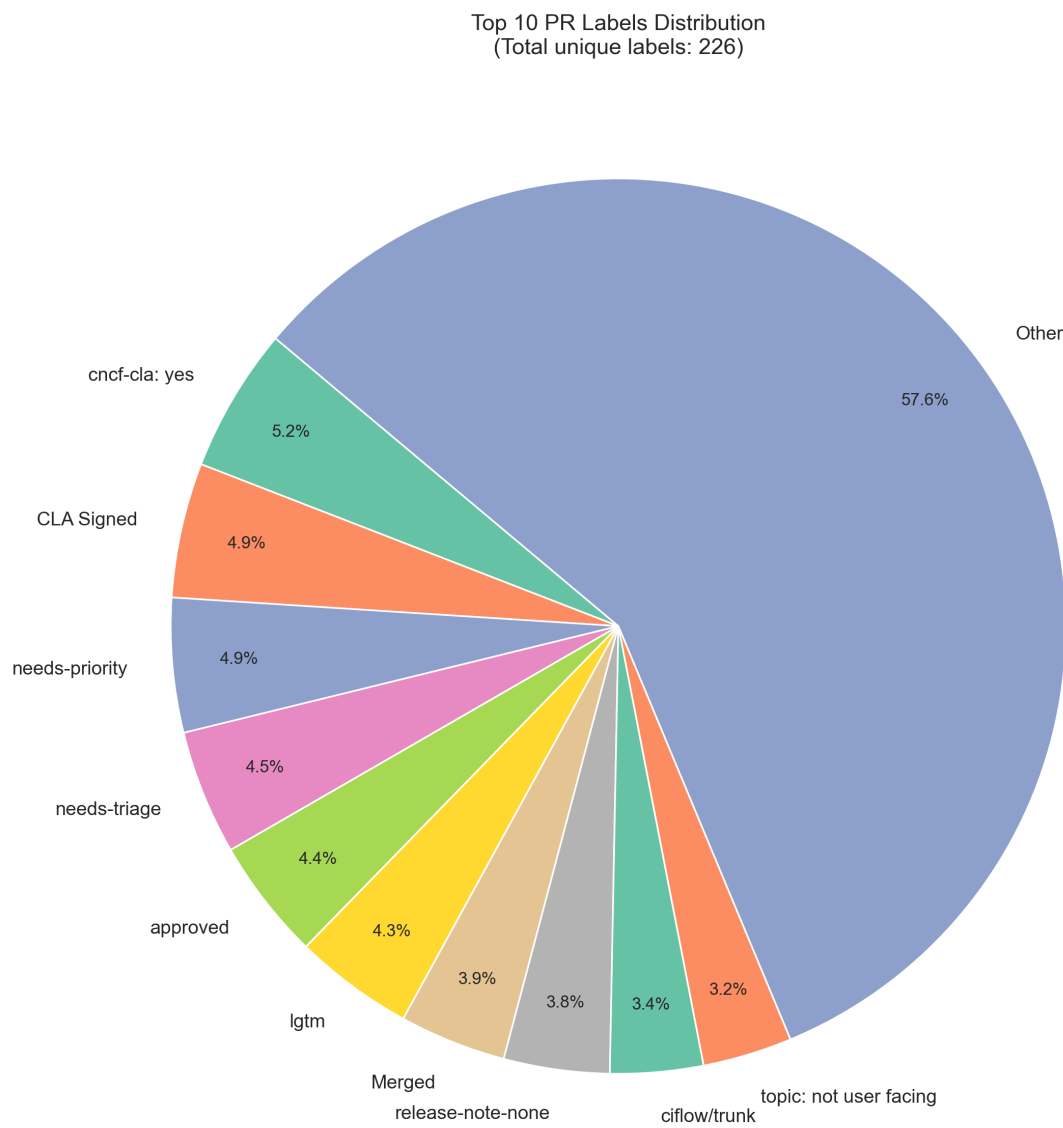


图 3: Top-10 PR 标签分布饼图

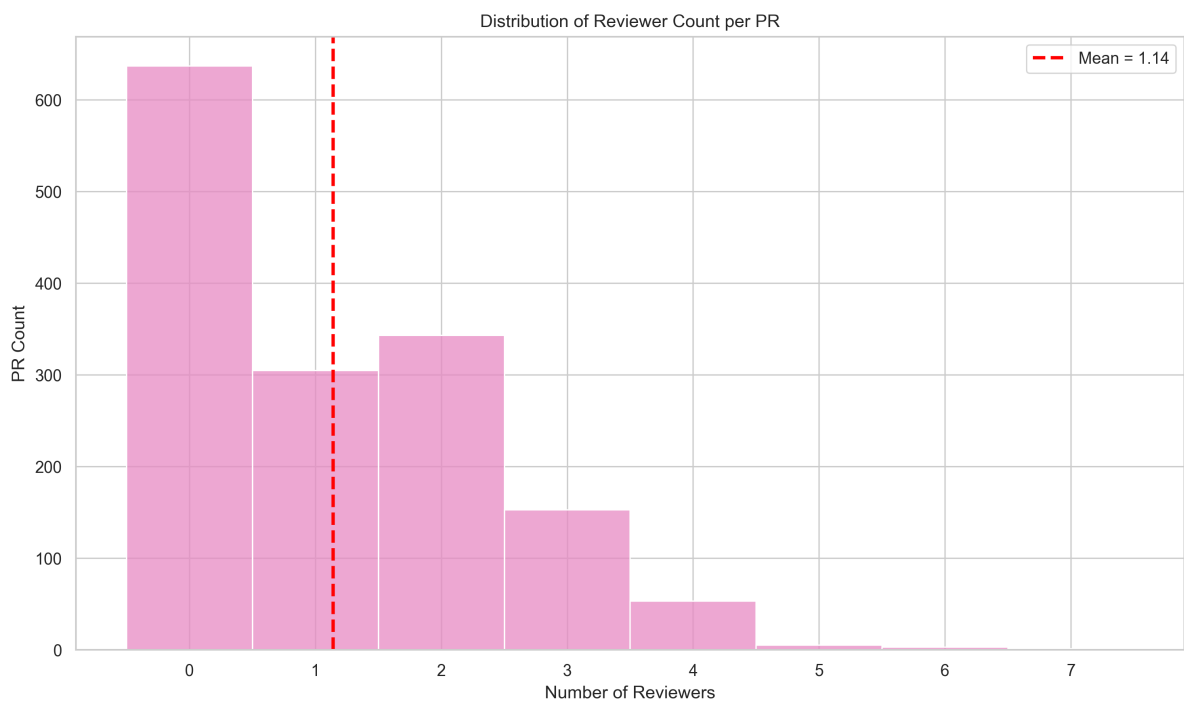


图 4: Reviewer 数量分布

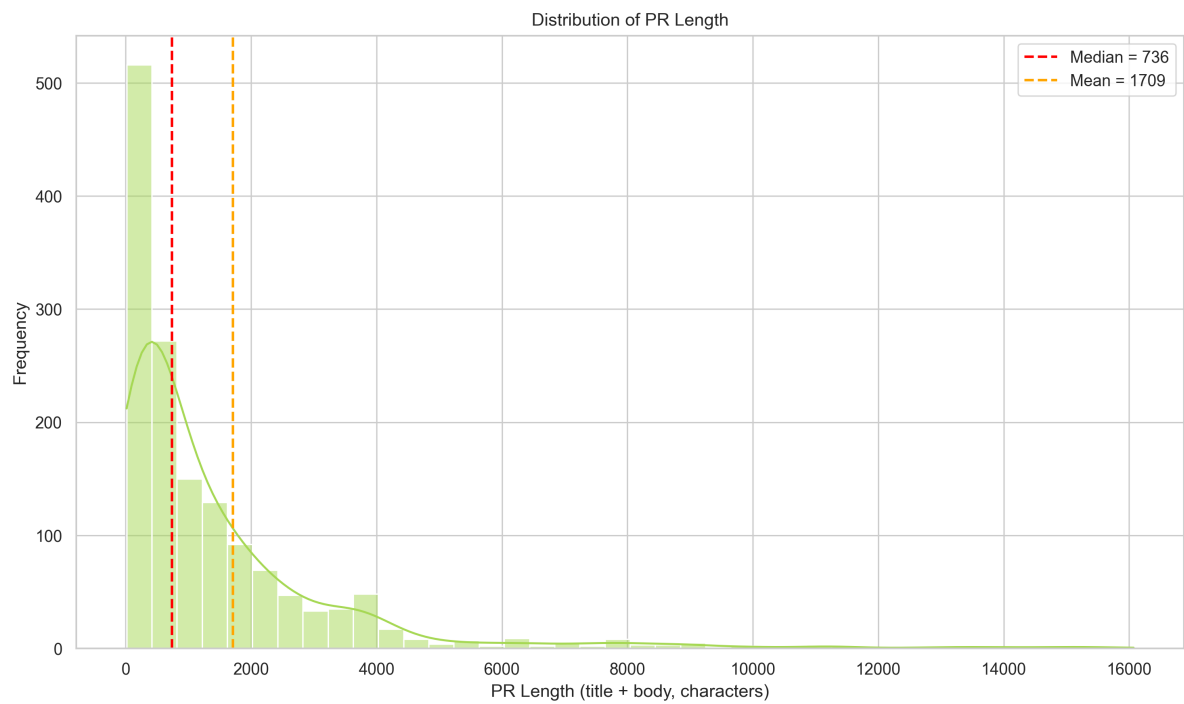


图 5: PR 长度分布直方图

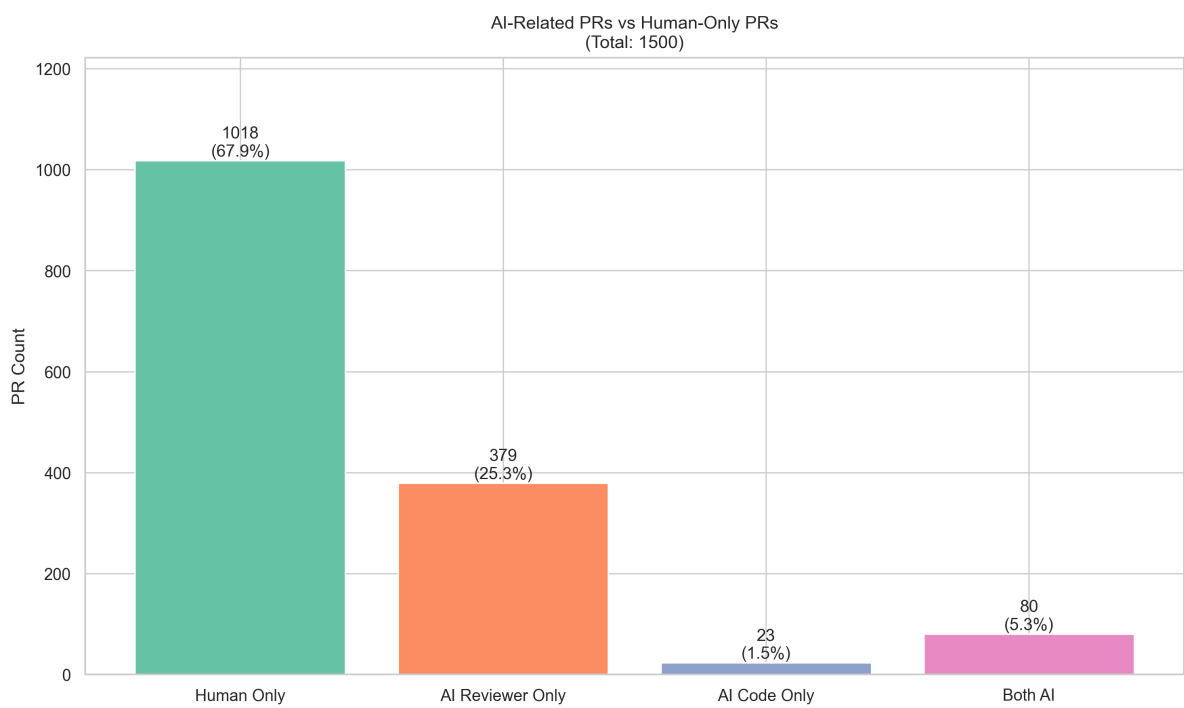


图 6: AI 相关 PR 与纯人类 PR 数量对比

四、实验中遇到的问题总结

要求：主要阐述实验过程中遇到的问题如何解决的。

4.1 Pull Request 与 Commit 的区别

Pull Request 是 GitHub 工作流中的一个高层概念，代表一个功能分支向目标分支（如 main）的合并请求。它通常包含多个 Commit，并附有标题、描述、标签、Reviewer 分配、CI 检查结果等元信息。而 Commit 是 Git 版本控制的基本单位，记录一次代码快照的变更。PR 是“请求合并一组 Commits”，而 Commit 是“单次变更的原子记录”。在实际开发中，一个 PR 往往包含多个迭代的 Commit，经过 Review 后可能通过 Squash Merge 合并为单个 Commit。

4.2 Code Review 为何能提高软件质量

Code Review 通过以下机制提高软件质量：（1）**知识共享**：团队成员通过审查了解代码变更，减少知识孤岛；（2）**缺陷发现**：审查者可能发现作者忽略的逻辑错误、边界条件和安全隐患；（3）**代码规范**：确保代码风格和架构决策符合团队标准；（4）**可维护性**：促使开发者编写更清晰、更易理解的代码，因为知道会被审查。我们的数据也支持这一点——合并的 PR 修改文件更少、描述更清晰，说明审查过程筛选出了高质量的变更。

4.3 Merge Prediction 的实际应用场景

Merge Prediction 可应用于：（1）**CI/CD 资源优化**：优先为高合并概率的 PR 分配 CI 资源；（2）**开发者辅助**：在 PR 提交前给出改进建议，提高合并成功率；（3）**项目管理**：预测 PR 的审查周期，帮助团队合理分配 Review 资源；（4）**开源社区健康度分析**：通过合并率评估项目的审查效率和社区活跃度。

4.4 AI Reviewer 与 Human Reviewer 的差异

从数据来看，AI Reviewer 占比达 35.4%，说明 AI 辅助代码审查已相当普遍。可能的差异包括：（1）**审查速度**：AI 可以秒级完成审查，人类需要数小时到数天；（2）**关注点**：AI 更多关注语法、风格和安全漏洞，人类更关注业务逻辑和设计决策；（3）**上下文理解**：AI 对项目全局架构和业务背景的理解有限，人类审查者具备领域知识；（4）**一致性**：AI 在不同时间给出一致的反馈，人类可能受情绪和疲劳影响。

4.5 数据集类别不平衡问题

数据集存在一定的类别不平衡（合并率 63.2%），不平衡程度较轻，由各仓库自然合并率差异造成。在后续的 Merge Prediction 任务中，需要注意：（1）**按仓库分层评估**：PyTorch（4.7%）和 TensorFlow（94.7%）的极端合并率可能导致仓库级别的性能差异；（2）**使用合适的评估指标**：不仅看准确率，还应关注 F1-Score、AUC-ROC 和 PR-AUC；（3）**可考虑分层采样**：在训练时按仓库和合并状态分层，确保模型学到跨仓库的泛化能力。

4.6 实验中遇到的技术问题

（1）**GitHub API 限速**：匿名访问仅 60 次/小时，需要使用 Personal Access Token 将限制提升至 5000 次/小时。（2）**数据采集速度**：初始串行方案每条约 8 秒，改为 3 线程并行 + PR 内部 5 路并行后，速度提升约 10 倍至 0.8 秒/条。（3）**PyTorch 仓库合并率为 0%**：由于按更新时间降序排列，近期的 300 条 PR 均为尚未合并的开放状态，证明了采样策略对数据分布的影响。（4）**修改函数提取**：从 Unified Diff 中提取函数名依赖正则表达式，对某些语言（如 C++ 模板）的支持不够完善。

五、实验体会

通过本次实验，我对 GitHub Pull Request 的数据结构有了系统的认识。从 API 数据采集到特征工程再到统计分析，完整地体验了数据科学项目从 0 到 1 的过程。

主要的收获包括：（1）理解了 PR、Review、Commit 三者之间的层级关系和数据模型；（2）掌握了 GitHub REST API 的分页、限速和认证机制；（3）学会了设计支持断点续传的数据采集流水线，这在处理大规模数据时非常重要；（4）通过特征分析发现了一些有趣的模式——合并的 PR 修改文件更少、描述更简洁，这印证了“小而精”的开源贡献原则。

本次构建的数据集将作为后续实验二至七的基石，为传统机器学习、深度学习和 LLM 方法在代码审查任务上的对比研究提供数据支撑。

指导教师评语：

日期：